

Ansible for z/OS

Working with DFSORT





Z/OS SORT UTILITY

Many batch processes involve combining inputs from various sources, consolidating and normalizing the data, formatting the data into a common format, sorting it, and loading it into a database or data warehouse.

This processing pattern is called Extract-Transform-Load, or ETL.

On z/OS, some ETL processing is handled by custom-written application programs and some is handled by IBM utilities.

One of the more common and complicated of these is the sort/merge utility.

Due to its long history and numerous enhancements, always with backward compatibility in mind, there are three entry points: SORT, DFSORT, and ICEMAN.

ICEMAN is the recommended entry point to call when running the utility. The program name is DFSORT.

It is possible to run ICEMAN with Ansible, but you have to understand the control statements for DFSORT. Ansible doesn't provide a convenient abstraction.



Z/OS SORT UTILITY

```
//SORTEXT3 EXEC PGM=ICEMAN
//SYSOUT DD SYSOUT=*
//SORTIN DD DSN=EMPLOYEE.DATA,DISP=SHR
//SORTOUT DD SYSOUT=*
//SYSIN DD *
  INCLUDE COND=(1,4,CH,EQ,C'EAST')
  SORT FIELDS=(1,3,CH,A,5,5,CH,D)
  OUTREC FIELDS=(1,3,X'40',20,21,X'40',50,6,PD,TO=ZD,X'40',C'$')
  SUM FIELDS=(10,8,PD,NONE)
/*
```

This is not a tested, working example. It's meant to give you a sense of what the DFSORT control statements look like.

You can probably guess that INCLUDE is defining some sort of input record filtering criteria; SORT is specifying the positions in each record to sort on, and whether to sort ascending or descending; OUTREC is probably for reformatting the records; and SUM pertains to counting something in the data. But the details are arcane, and you will have to refer to IBM's documentation on DFSORT.

When you use Ansible with module `zos_mvs_raw`, you have to drop something like this directly into your task, under `dd_name: sysin`. There are no short-cuts, unfortunately.

MAINFRAME VS. UNIX APPROACH

```
cat input.txt \  
| grep '^AB' \  
| sort -k2,2 \  
| uniq \  
| awk '{ printf "%s,%s,%s\n", $1, $2, $5 }'
```

The mainframe approach for utility programs is to have a single executable perform all the various processing options and to control the program through statements like those in the previous example, under the SYSIN DD.

You may be more accustomed to the Unix approach, in which each program performs a single function, and solutions are built by piping programs and passing the data along via stdin and stdout; something like the crude example above.

It may be frustrating to work with the z/OS utilities until you get used to this difference.

DFSORT CONTROL STATEMENTS

```
//IBMUSERS JOB , 'SORT 1',
//          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
//          NOTIFY=&SYSUID
//*****
//* Simple sort example
//*****
//SORT1     EXEC PGM=ICEMAN
//SYSOUT    DD  SYSOUT=*
//SORTIN     DD  *
GEORGIA      ATLANTA
INDIANA      INDIANAPOLIS
ALABAMA      MONTGOMERY
NEW JERSEY   TRENTON
/*
//SYSIN      DD  *
  SORT FIELDS=(1,14,CH,A)
/*
//SORTOUT    DD  SYSOUT=*
```

```
***** TOP OF DATA ***
ALABAMA      MONTGOMERY
GEORGIA      ATLANTA
INDIANA      INDIANAPOLIS
NEW JERSEY   TRENTON
***** BOTTOM OF DATA *
```

SORT FIELDS specifies one or more fields in the input record on which to sort. Each field is described by four values: starting position relative to 1, length in bytes, data type (here: CH for character), ascending or descending (A or D).

DFSORT CONTROL STATEMENTS

```
//IBMUSERS JOB ,'SORT 2',
//          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
//          NOTIFY=&SYSUID
//*****
//* Sort example with two fields
//*****
//SORT2     EXEC PGM=ICEMAN
//SYSOUT    DD  SYSOUT=*
//SORTIN    DD  *
ALABAMA      HUNTSVILLE
GEORGIA      ATLANTA
INDIANA      INDIANAPOLIS
GEORGIA      COLUMBUS
NEW JERSEY   JERSEY CITY
ALABAMA      MONTGOMERY
NEW JERSEY   TRENTON
INDIANA      SOUTH BEND
/*
//SYSIN     DD  *
  SORT FIELDS=(1,14,CH,A,15,20,CH,D)
/*
//SORTOUT   DD  SYSOUT=*
```

```
***** TOP OF DATA ***
ALABAMA      MONTGOMERY
ALABAMA      HUNTSVILLE
GEORGIA      COLUMBUS
GEORGIA      ATLANTA
INDIANA      SOUTH BEND
INDIANA      INDIANAPOLIS
NEW JERSEY   TRENTON
NEW JERSEY   JERSEY CITY
***** BOTTOM OF DATA *
```

We added a second sort field, descending. This field is sorted within each unique value of the first field. So we expect to see the states listed in ascending order, and cities within each state listed in descending order.

DFSORT CONTROL STATEMENTS

```
//IBMUSERS JOB , 'SORT 3',
//          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
//          NOTIFY=&SYSUID
//*****
//* Sort example with OUTREC
//*****
//SORT2     EXEC PGM=ICEMAN
//SYSOUT    DD  SYSOUT=*
//SORTIN    DD   *
ALABAMA      HUNTSVILLE      AL
GEORGIA      ATLANTA           GA
INDIANA      INDIANAPOLIS      IN
GEORGIA      COLUMBUS          GA
NEW JERSEY   JERSEY CITY       NJ
ALABAMA      MONTGOMERY         AL
NEW JERSEY   TRENTON           NJ
INDIANA      SOUTH BEND        IN
/*
//SYSIN     DD   *
  SORT FIELDS=(1,14,CH,A,15,20,CH,D)
  OUTREC BUILD=(35,2,C' ',15,20)
/*
//SORTOUT   DD  SYSOUT=*
```

```
***** TOP OF DATA ***
AL MONTGOMERY
AL HUNTSVILLE
GA COLUMBUS
GA ATLANTA
IN SOUTH BEND
IN INDIANAPOLIS
NJ TRENTON
NJ JERSEY CITY
***** BOTTOM OF DATA *
```

Now we use OUTREC to reformat the output records. We pick up the state code from positions 35-36 in the input record, and place it into positions 1-2 of the output record. Then we put a space in position 3. Finally, we put the city name in positions 4-23. There are many options for OUTREC and you will never memorize them. Also, you will only have to work with DFSORT occasionally, so you will forget details. The documentation is your friend.

DFSORT CONTROL STATEMENTS

```
//IBMUSERS JOB , 'SORT 4',
//          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
//          NOTIFY=&SYSUID
//*****
//* Sort example with OUTREC
//*****
//SORT2     EXEC PGM=ICEMAN
//SYSOUT    DD  SYSOUT=*
//SORTIN     DD   *
ALABAMA      HUNTSVILLE      AL
GEORGIA      ATLANTA           GA
INDIANA      INDIANAPOLIS      IN
GEORGIA      COLUMBUS          GA
NEW JERSEY   JERSEY CITY       NJ
ALABAMA      MONTGOMERY        AL
NEW JERSEY   TRENTON           NJ
INDIANA      SOUTH BEND        IN
/*
//SYSIN      DD   *
  SORT FIELDS=(1,14,CH,A,15,20,CH,D)
  OUTREC BUILD=(35,2,C' ',15,20,C' DATE: ',DATE3-1)
/*
//SORTOUT    DD  SYSOUT=*
```

```
***** TOP OF DATA ***
AL MONTGOMERY      DATE: 2026027
AL HUNTSVILLE    DATE: 2026027
GA COLUMBUS        DATE: 2026027
GA ATLANTA         DATE: 2026027
IN SOUTH BEND      DATE: 2026027
IN INDIANAPOLIS    DATE: 2026027
NJ TRENTON         DATE: 2026027
NJ JERSEY CITY     DATE: 2026027
***** BOTTOM OF DATA *
```

In this example we insert the text ' DATE: ' and the system date for yesterday in IBM's quasi-Julian format, representing the year and the day of the year.

DFSORT CONTROL STATEMENTS

```
//IBMUSERS JOB , 'SORT 5',
//          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
//          NOTIFY=&SYSUID
//*****
//* Sort example with OUTREC, remove duplicate records
//*****
//SORT2     EXEC PGM=ICEMAN
//SYSOUT    DD  SYSOUT=*
//SORTIN    DD  *
ALABAMA      HUNTSVILLE      AL
GEORGIA      ATLANTA           GA
INDIANA      INDIANAPOLIS      IN
INDIANA      INDIANAPOLIS      IN
GEORGIA      COLUMBUS          GA
NEW JERSEY   JERSEY CITY       NJ
ALABAMA      MONTGOMERY        AL
NEW JERSEY   TRENTON           NJ
GEORGIA      ATLANTA           GA
INDIANA      SOUTH BEND        IN
/*
//SYSIN     DD  *
  SORT FIELDS=(1,14,CH,A,15,20,CH,D)
  OUTREC BUILD=(35,2,C' ',15,20,C' DATE: ',DATE3-1)
  SUM FIELDS=NONE
/*
```

```
***** TOP OF DATA ***
AL MONTGOMERY      DATE: 2026027
AL HUNTSVILLE    DATE: 2026027
GA COLUMBUS        DATE: 2026027
GA ATLANTA         DATE: 2026027
IN SOUTH BEND      DATE: 2026027
IN INDIANAPOLIS    DATE: 2026027
NJ TRENTON         DATE: 2026027
NJ JERSEY CITY     DATE: 2026027
***** BOTTOM OF DATA *
```

SUM FIELDS=NONE removes duplicate records, where records are exact duplicates. We can also identify logically-duplicate records based on their contents. We'll skip that for now – there are just too many options to cover one by one in this way.

LAB 27: CONCATENATE AND SORT DATA SETS

Follow the instructions for Lab 27 in labs/labs.md in the course repo.

WHAT YOU KNOW AND WHAT YOU CAN DO

- You can write simple control statements for DFSORT
- You can execute DFSORT (ICEMAN) from Ansible

WHAT'S NEXT

- Provision data sets for an application developer